

bus is a multibit interface to which drivers are connected through their output buffers. Inputs of various blocks can be directly interfaced to the data bus. Here the width of the data bus is a generic called 'width.'

Two multibit 2:1 multiplexers are used: One is used to select between Data In and the computed result from data path output, to be written into register file. The other is used to select one of the outputs from ALU and multiplier as data path output.

Register file is a set of local registers used by data path. There is one input and two output ports. The two outputs are the two input operands to the ALU or the multiplier. The input is used for writing to one of the registers. Write register is selected by the write address and two read registers are selected by the two read addresses.

One of the two operands passes through a barrel shifter. Barrel shifter shifts the bits by specified count to left or right. The new bits on the left or the right are 0s or rotate around of the word.

The two operands beyond barrel shifter are passed on to the ALU and the multiplier as inputs. The ALU is capable of one of the eight operations. The multiplier multiplies two operands and provides the multiplexed output.

The output of the second multiplexer moves to the output universal shifter. The universal shifter is capable of bit-wise left and right shift, serial-to-parallel conversion and multiplexing the output with a second input word. Output buffer at the output of data path provides an interface to the bus. This is required in case multiple data paths drive the output bus.

The various parts included in Fig. 2 are discussed along with VHDL codes in the following sections.

Synthesis tool

Xilinx Vivado is used as the synthesis tool (Fig. 3). It manages design,



Fig. 3: Synthesis tool and FPGA board

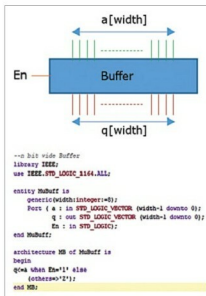


Fig. 4: VHDL code for multibit buffer

synthesis, implementation and device configuration. The flow is as follows:

1. Create a project and select Artix 7 FPGA that you wish to use
2. Add source files for designs
3. Simulate designs and validate results
4. Synthesise designs
5. Specify the input/output (I/O) constraints and generate xsd files
6. Re-synthesise designs
7. Implement designs
8. Generate bit (configuration) files for the device
9. Open (connect through UART) the device connection
10. Configure the device using bit files
11. Validate designs using switches and LEDs of the board



Fig. 5: Simulation and implementation of buffer

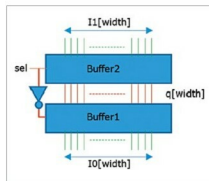


Fig. 6: Multibit 2:1 multiplexer

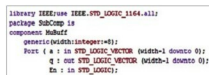


Fig. 7: Package definition with buffer as component

Multibit buffer

Fig. 4 shows a multiple-bit buffer along with MuBuff.vhd code. The output assumes the same value as the input if En is high. Else, output bits are high impedance. You can see the 1's and 0's bits on the LEDs. Fig. 5 shows its implementation on the Nexys 4 board.

Multiplexer

The multibit 2:1 multiplexer can be designed using two multibit buffers. Fig. 6 shows a 2:1 multiplexer. Fig. 7 shows package definition with buffer as component. The hierarchical style code for the multiplexer is shown below (MUXnbit2to1.vhd.):

```

-- n bit 2:1 mux using tristate buffers
-- MUXnbit2to1.vhd
  
```